Open in app ↗

Search Write

Debugging containerd



Sam Lockart · Follow
Published in Zendesk Engineering · 6 min read · Jun 8, 2022

37

containerd is the container runtime interface (CRI) that (will soon) power Zendesk.

A new container runtime means new debugging techniques are required.

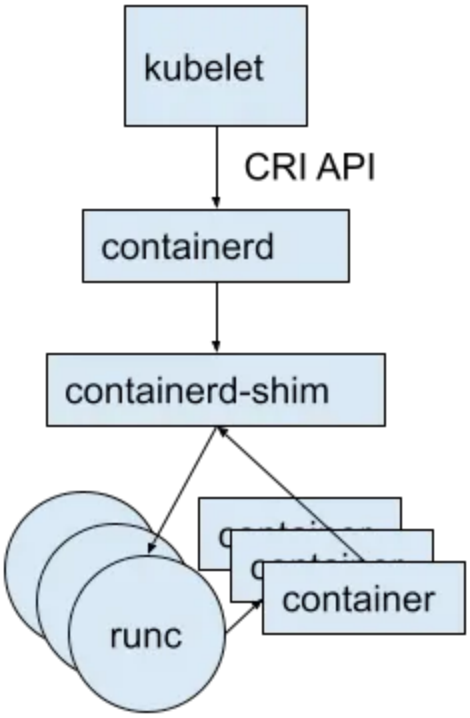
Many of the techniques described here were learnt from necessity while debugging an issue we observed in our staging environment while soaking containerd 1.6.2.

. . .

The containerd stack comprises of a few different binaries that all perform a certain set of tasks.

To properly debug containerd, we need to know how to inspect each component.

The delve debugger is very useful, but to debug any of the *containerd* stack you must compile the binary with debug symbols. This can be time consuming and if the issue is transient, you won't have time.



A very basic overview of the stack we'll be debugging

When a pod is to be created for example, the *kubelet* talks to *containerd* using the CRI interface. *Containerd* will then fork/exec a *containerd-shim* instance which in turns invokes the *runc* binary. It is up to *runc* to create, run, delete and clean up the container.

. . .

1. Kubelet

```
$ journalctl -xe --unit kubelet # view logs for kubelet
$ systemctl status kubelet # view the status of the service from
```

```
systemd's perspective
$ kubectl get pods -o wide | grep $NODENAME
```

Debugging containerd from the point of the kubelet is quite simple. Namely we want to check the kubelet logs to see if there are any glaring issues.

Note that issues from lower in the stack (e.g. runc) will hopefully bubble up to the kubelet logs. If a container cannot be created due to an issue with runc, the kubelet will probably complain about deadlines being missed for pod creation.

Confirm that kubelet is configured to actually use containerd. *The flags* `–container-runtime=remote` and `--container-runtime-endpoint=unix:///run/containerd/containerd.sock` need to be set.

. . .

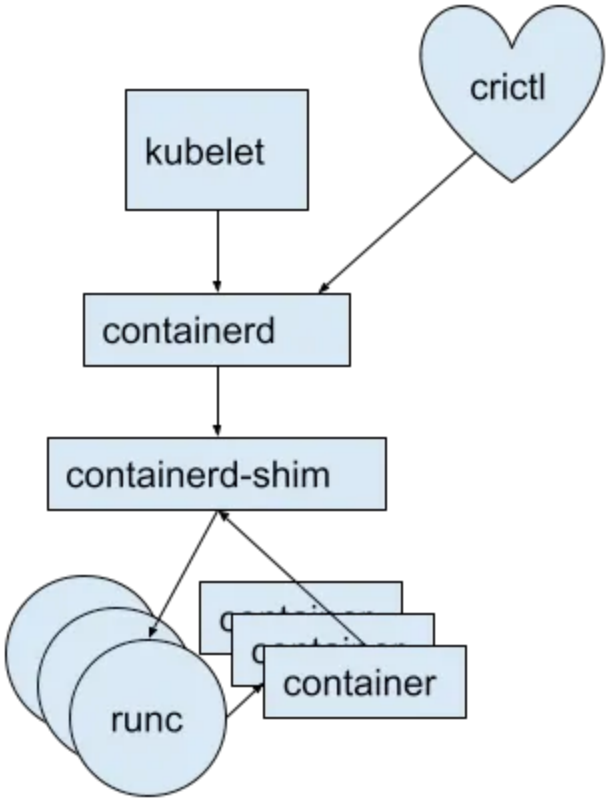
2. Containerd

```
$ systemctl status containerd
$ journalctl -xe --unit containerd
$ containerd config dump # see the output of the final main config
with imported in subconfig files
$ diff -y <(containerd config default) <(containerd config dump)
$ crictl
$ systemd-cgls # view the cgroups under `containerd.service`
$ dpkg -L containerd.io # list everything installed by containerd
package (note that we manually overwrite some of these files)
```

First thing to check, similar to kubelet, are the service status and the logs.

Going further in, we can look at the config file which *containerd* is using, and can even compare it with the default config. If anything catches your eye, whether it be a log line or some unexpected configuration, take a look through the issues.

The *crictl* tool is very useful as it lets you interact with the container runtime in the same way *kubelet* does. See https://github.com/kubernetes/cri-api.



crictl uses the CRI to interface with containerd and can be used without any Kubernetes components

```
$ crictl pods # list pods - compare with kubectl get pods
$ crictl inspectp # inspect a pod (different from crictl inspect)
$ crictl info # view CRI runtime info
$ crictl stats # does everything look normal?
$ crictl --debug stopp # [sic] stop a pod and print debug logs
(useful if a pod is stuck terminating)
```

Enabling debug on *containerd* is as simple as editing the *containerd* config to enable the socket.

```
[debug]
  address = "/run/containerd/debug.sock"
#  uid = 0
#  gid = 0
# supported values [trace, debug, info, warn, error, fatal, panic]
# https://github.com/containerd/containerd/blob/v1.6.1/cmd/containerd/command/main.go#L89
  level = "debug"
# https://github.com/containerd/containerd/blob/v1.6.1/log/context.go#L45-L49
  format = "json"
```

The config for containerd is generally found at /etc/containerd/config.toml

This allows you to do things like dump all the goroutines that *containerd* has started, get traces etc.

```
(sam.lockart@zendesk.com) root@ip-172-30-229-138:/proc/5807# ctr pprof
NAME:
   ctr pprof - provide golang pprof outputs for containerd

USAGE:
   ctr pprof command [command options] [arguments...]

COMMANDS:
   block         goroutine blocking profile
   goroutines    dump goroutine stack dump
   heap          dump heap profile
   profile       CPU profile
   threadcreate  goroutine thread creating profile
   trace         collect execution trace

OPTIONS:
   --debug-socket value, -d value  socket path for containerd's debug server (default: "/run/containerd/debug.sock")
   --help, -h                      show help
```

• • •

3. containerd-shim

```
$ ps aux --forest | less # scroll down to a containerd-shim-runc-v2
process and take a look
# for pid in $(pgrep containerd-shim-runc-v2); do kill -USR1 $pid;
done # dump every shim processes goroutine to the containerd journal
$ journalctl -xe --unit containerd | grep shim > somefile # run after
dumping goroutines to save the output
```

```
# strace -p $CONTAINERD_PID # trace syscalls, normally it will be
sleeping though
```

The debugging methods here are probably a bit overkill, but if you have gone down this article from the *kubelet*, and have now reached here, it might be worth it!

Each shim has responsibility for a single pod. If you run the `ps aux --forest` command above, you should see:

- the almighty pause container
- the process defined in the container's entrypoint



The forest flag shows the relationship between parent and child processes

The example above is a bit more complicated, we are running the pause container, plus *sh* which has gone and run a bash script. This then has run *aws-k8s-agent* which is presumably being piped into tee. Finally, this is all being piped through *logrecycler*.

The shim also acts as a sub reaper, i.e. it will reap zombie processes. If the shim process goes away, the processes underneath are not stopped, instead they are orphaned to *containerd*.

Zombie process - Wikipedia



> On Unix and Unix-like computer operating systems, a zombie process or defunct process is a process that has completed...
>
> en.wikipedia.org



If for whatever reason *containerd-shim* isn't running the pause container, there may be an issue creating the pod sandbox. Do a sanity check of the node(s) in question and look at performance (a great article which inspired the format of this write up).

If you suspect something is wrong with *containerd-shim*, e.g. it is consuming lots of resources (they normally will be voluntarily sleeping) there are a number of things you can do. Namely, dumping the goroutines.

. . .

4. runc / container

```
# alias runk=runc --root /run/containerd/runc/k8s.io/ # alias for
setting the root directory
# runk events # view container events
# runk state # view container state
```

This is where the rubber hits the road: the next steps for debugging depend on the type of problem you are trying to debug. Where to specifically debug at this level is very much dependent on the problem you are trying to debug. Next, I will provide some insight into how *runc* works and where you can look for further troubleshooting.

When a container is created as part of some Kubernetes process, the container is created under the `http://k8s.io` namespace (this isn't a

Kubernetes namespace, rather it is a place where all of the container runtime pod state relating to Kubernetes is stored).

To run the various *runc* subcommands, you often need to provide a `--root` flag. This will be the state directory which is at `/run/containerd/runc/k8s.io/`.

When *runc* starts a container, it is actually a number of different actions:

- `runc create` creates a container and starts a bare `runc init` process in it. This `runc init` then waits for the exec FIFO file to be opened on the other side, as a mechanism of synchronization. Once opened, it writes a `0` byte to it, and proceeds to execute the container's entry-point.
- `runc start` actually starts that container (by opening the exec FIFO file and reading the data from it), signalling `runc init` that it should proceed.

Source: Kir Kolyshkin @ https://github.com/opencontainers/runc/issues/3448

If there were a problem and for example, *containerd* was in a deadlock like we have seen before:

- the `runc init` processes would be eternally waiting for `runc start`
- the `runc init` processes would keep creating every few minutes
- pods won't be able to run
- pods won't be able to terminate

. . .

Thanks

A big thank you to Fu Wei and the containerd team who helped us debug and ultimately fix the deadlock issue we experienced in our staging environment

. . .

Appendix

Core dumps can be created by running *gdb*, attaching to the process and running generate-core-dump. I suggest gzipping the coredumps before transferring these (they are quite verbose, but a good candidate for compression).

These core dumps can be used with delve, but as mentioned above, the binaries must have debug symbols!

Note that attaching a debugger to any running process may slow it down significantly. This is because debuggers like *delve* or *gdb* insert hooks into various parts of the process and imported libraries. These hooks are extra code paths that incur a non-trivial increase to execution time. A process that may be already suffering performance issues for example will hindered more so.

Be careful about running these in production.

Links and further reading

- https://iximiuz.com/en/posts/implementing-container-runtime-shim/
- https://netflixtechblog.com/linux-performance-analysis-in-60-000-milliseconds-accc10403c55

Kubernetes Containerd Debugging Golang Delve

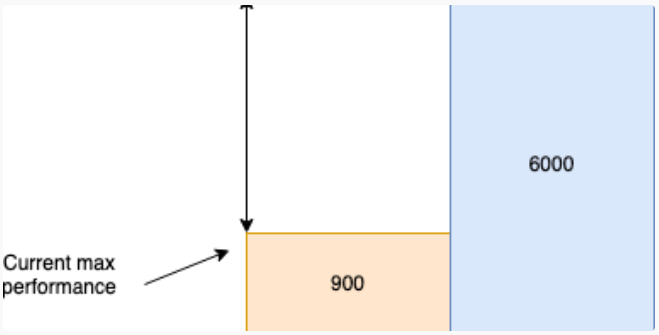


Written by Sam Lockart

32 Followers · Writer for Zendesk Engineering

Follow

More from Sam Lockart and Zendesk Engineering





Sam Lockart in Zendesk Engineering

etcd: getting 30% more write/s

Undertaking performance analysis on our etcd clusters, I ended up with a 30%...

8 min read · Jan 24

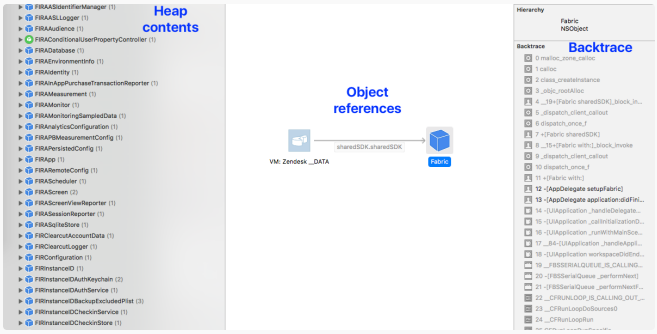
Mahdi Karabiben in Zendesk Engineering

dbt at Zendesk—Part I: Setting foundations for scalability

An overview of ED&A's data transformation patterns and our path towards a dbt-based...

10 min read · May 22

50 2



Pete Smith in Zendesk Engineering

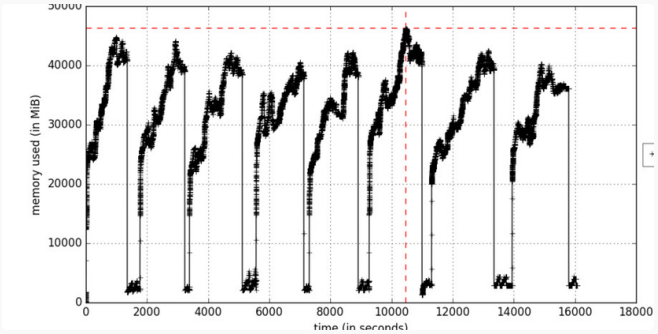
iOS — Identifying Memory Leaks using the Xcode Memory Graph…

In this short post I describe,

· 3 min read · Apr 20, 2017

1.4K 3

134



Wai Chee Yau in Zendesk Engineering

Hunting for Memory Leaks in Python applications

We use Python a fair bit at Zendesk for building machine learning (ML) products. On…

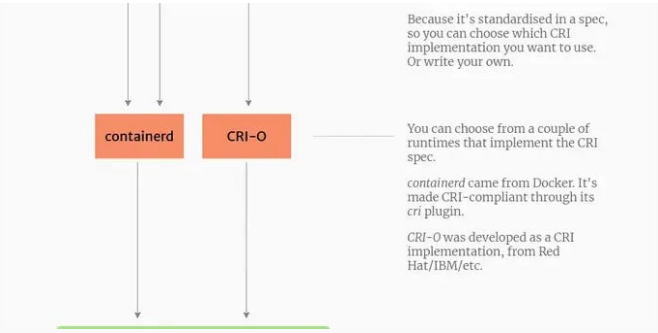
4 min read · Feb 12, 2019

693 2

See all from Sam Lockart

See all from Zendesk Engineering

Recommended from Medium



Btech Engineering

Dedicated CPU Kubevirt

enable decicated cpu for VM Kubevirt

3 min read · May 12

2

Vineet Kumar

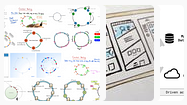
The differences between Docker, containerd, CRI-O and runc

Containers aren't tightly coupled to the name Docker. You can use other tools to run...

6 min read · Jun 24

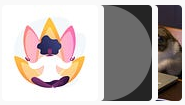
13

Lists



General Coding Knowledge

20 stories · 482 saves



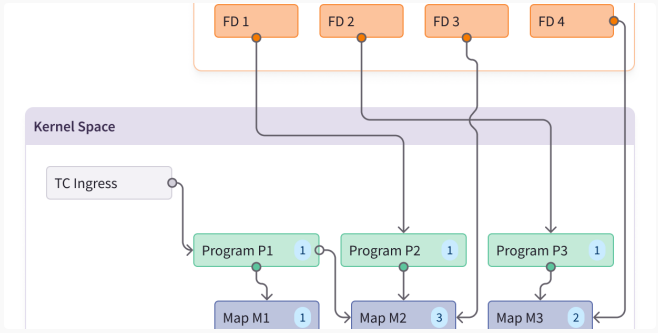
Stories to Help You Grow as a Software Developer

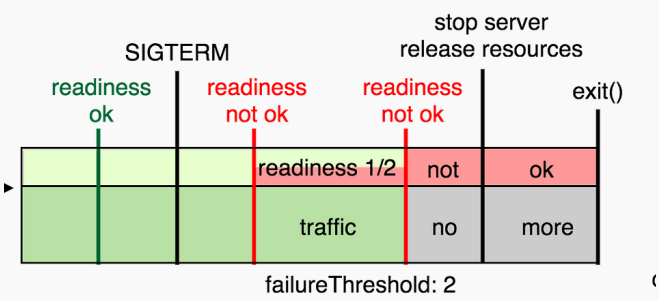
19 stories · 491 saves



Natural Language Processing

753 stories · 338 saves





Daniel Xu in Aviatrix Engineering

BPF tailcalls

One of the products I work on is a firewall that implements L3/L4 enforcement using BPF....

7 min read · Oct 11

19

Arkadev Banerjee in CityMall Engineering

Effective GoLang Graceful Shutdown in Kubernetes:...

The above picture narrates the Kubernetes traffic shifting story.

4 min read · Jul 21

31



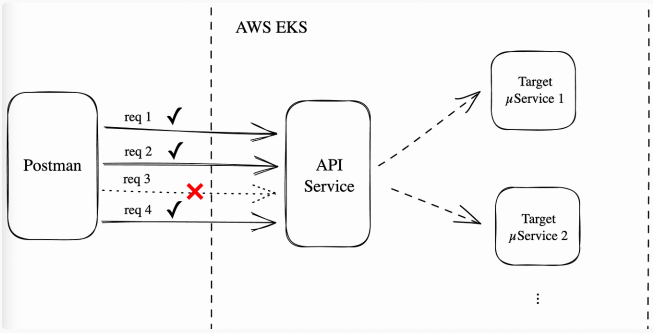
Redouane Achouri in Better Programming

Hunting for a Memory Leak in a Node.js Application on Kubernete...

Debug Node.js remotely in a production Kubernetes environment, with no additional...

6 min read · May 2

101



Sam Rajaei in Binome

Connection Pooling and Intermittent Failures in K8s

Intro

8 min read · Jun 5

57 1

See more recommendations